Infotact Solutions - Data Intelligence Unit

Official Memo

To: Data Science Engineering Track (Cohort Zeta)
From: Head of AI & Analytics
Date: December 1, 2025
Subject: Q4 Production Machine Learning Assignments: Building Scalable, Reproducible, and Interpretable AI Systems-----Executive Summary & Philosophy

Team,

Welcome to the Capstone Phase of the Data Science Engineering Track. This quarter is dedicated to transitioning your theoretical knowledge into **production-grade AI systems.** At Infotact Solutions, Data Science is fundamentally an engineering discipline. It is *not* about creating isolated, non-maintainable code snippets or plotting aesthetically pleasing graphs in a single Jupyter Notebook that resides solely on your local machine.

Our core mandate is the development of **Scalable, Reproducible, and Interpretable AI Systems** that deliver measurable business value. Over the next four weeks, you will be tackling four high-impact, real-world assignments. These projects span the four pillars of modern Artificial Intelligence, ensuring you gain comprehensive experience across the spectrum of enterprise AI challenges:

1. **Tabular Prediction (IoT/Time-Series):** Predictive Maintenance.
2. **Computer Vision (Retail/Manufacturing):** Visual Quality Control.
3. **Natural Language Processing (FinTech/Legal):** Intelligent Document Parsing.
4. **Big Data Processing (Telecom):** Scalable Churn Prediction using Distributed Compute.

We operate strictly on a **4-week sprint cycle**. Success is defined by the delivery of a **Production-grade code structure** (modular `.py` files, classes, and functions, not monolithic notebooks) and the rigorous application of **MLOps principles** (versioning, testing, and deployment readiness).-----Contents - Project and Architecture Index

1. **Tech Stack - Production AI Architecture Deep Dive**
2. **Project 1: Manufacturing - Predictive Maintenance with Explainable AI (Tabular/IoT)**
3. **Project 2: FinTech - Intelligent Document Parsing (NLP/NER)**
4. **Project 3: Retail - Visual Quality Control System (Computer Vision)**
5. **Project 4: Telecom - Big Data Churn Prediction (Apache Spark/Distributed Compute)**

-----1. Tech Stack - Deep Explanation of the Production AI Architecture

We utilize a hybrid technology stack designed to maximize experimentation agility while guaranteeing deployment stability and performance at scale. Mastery of this toolset is non-negotiable.

Category	Primary Tools	Application and Rationale
Data Processing & Engineering	**Pandas & NumPy**	**In-Memory Operations:** Used for rapid Exploratory Data Analysis (EDA), feature prototyping, and high-performance vectorization on datasets that fit within single-machine memory (Gigabyte scale).
	Apache Spark (PySpark)	**Distributed Compute:** Essential for handling data volume that exceeds the capacity of a single machine (Terabyte scale, common in Projects 4). Enables distributed ETL and large-scale aggregations.
Machine Learning Engines	**Scikit-Learn**	**Classical ML & Preprocessing:** The robust foundation for baseline models (Linear Regression, SVM, Random Forest) and managing complex preprocessing pipelines (StandardScaler, OneHotEncoder).
	XGBoost / LightGBM	**High-Performance Tabular Data:** The undisputed industry standard for achieving state-of-the-art accuracy on structured, tabular datasets via Gradient Boosting Decision Trees (GBDT). Crucial for Project 1 & 4.

	TensorFlow/Keras	**Deep Learning Framework:** Utilized for building and training complex neural networks required for unstructured data—specifically **Convolutional Neural Networks (CNNs)** for Vision (Project 3) and **Recurrent/Transformer models** for NLP (Project 2).
Development & MLOps	**Jupyter Notebooks**	**Strictly for EDA:** Their use is restricted to initial data investigation, visualization, and rapid prototyping of model features. They **must not** be used for final production code.
	PyCharm / VS Code	**Production Code IDEs:** Mandatory for writing modular, testable, and reusable production Python code, adhering to PEP 8 standards.
	MLflow (Bonus Implementation)	**Experiment Tracking & Versioning:** Highly encouraged for tracking metrics, logging hyperparameters, and managing model artifacts across different runs. Demonstrates true MLOps maturity.

-----2. Project 1: Industrial AI - IoT Predictive Maintenance Engine

Project Title: IoT Predictive Maintenance Engine
Product Brand Name: "FactoryGuard AI"

Use Case (Production Context): A large-scale manufacturing facility operates 500 critical robotic arms. Unplanned equipment failure results in downtime costing approximately **$10,000 per hour**. The objective is to construct a binary classification system that predicts equipment failure with a lead time of **24 hours** based on streaming sensor data (vibration, temperature,

pressure). This enables proactive, scheduled maintenance.Key Product Features & Deliverables:

- **Basic Requirement:** Implement a robust Data Cleaning process, calculate a Correlation Matrix, and establish a Logistic Regression Baseline model for comparison.
- **Deep Production Features:**
  - **Temporal Feature Engineering:** Creation of **"Rolling Window" statistics** (e.g., Mean Vibration over the last 1/4/8 hours, Exponential Moving Averages) to capture essential temporal trends and drift.
  - **Imbalance Handling:** Failures are inherently rare events (typically **<1%** of the dataset). The model must utilize techniques like **SMOTE (Synthetic Minority Over-sampling Technique)** or class weighting to prevent the model from trivializing the positive class.
  - **Explainable AI (XAI):** Mandatory use of **SHAP (SHapley Additive exPlanations)** to provide local interpretability. The model must explain *why* a specific machine is flagged (e.g., "Risk factor due to Temp > 80°C and Pressure Stability < 0.5"). This builds trust with engineering teams.

Implementation Details:

- **Core Stack:** Python $\rightarrow$ Pandas $\rightarrow$ Scikit-Learn $\rightarrow$ **XGBoost**.
- **Resources:** XGBoost documentation, `imbalanced-learn` library.

Week-Wise Implementation Plan:

Week	Focus Area	Tasks/Goals	Review/Validation
Week 1	**Data Engineering**	Goal: Transform Raw Sensor Logs (CSVs) into a Modeling Dataset. Tasks: Ingest, Handle missing values (e.g., Interpolation), Generate Lag Features (t-1, t-2) and Rolling Statistics.	Check for **"Data Leakage"** (Ensure no future information is used to predict the past).

Week 2	**Modeling & Hyperparameter Tuning**	Goal: Beat the simple baseline. Tasks: Train Random Forest. Train the high-performance **XGBoost** model. Use **RandomizedSearch CV** for hyperparameter optimization.	**Evaluation Metric:** Focus on **F1-Score** and **Recall** (minimizing false negatives), **NOT** simple Accuracy.
Week 3	**Interpretability & Trust (XAI)**	Goal: Establish model trustworthiness. Tasks: Calculate and analyze **SHAP values** for the test set. Create **Force Plots** and summary plots to understand feature impact on failure predictions.	Validate model logic against physical expectations (e.g., Is high temperature correlated with high risk?).
Week 4	**Deployment Wrapper**	Goal: Deliver a Model-as-a-Service endpoint. Tasks: Serialize the final model (using `joblib.dump`). Build a minimal **Flask API** that accepts new sensor readings (JSON) and returns the "Failure Probability" and the SHAP-based explanation.	API Latency Check (**Target: < 50ms** for real-time viability).

-----3. Project 2: FinTech (NLP) - Intelligent Document Parsing (NER)

Project Title: Automated Legal Entity Extractor
Product Brand Name: "LexiScan Auto"

Use Case (Production Context): A high-volume law firm must process thousands of unstructured PDF contracts daily. Manually extracting key entities like *Effective Dates*, *Party Names*, *Total Monetary Values*, and *Jurisdiction* is slow and error-prone. Standard Regular

Expressions (Regex) are too rigid. We require a **Named Entity Recognition (NER)** deep learning system.Key Product Features & Deliverables:

- **Basic Requirement:** Implement simple TF-IDF classification and preliminary Regex pattern matching for initial entity identification.
- **Deep Production Features:**
    - **Custom NER Model:** Training a state-of-the-art Deep Learning model (e.g., **Bi-Directional LSTM** or fine-tuning a small **BERT-based model**) tailored to the specific vocabulary of legal documents.
    - **Robust OCR Pipeline:** Integrating **Tesseract OCR** (or an equivalent) to convert scanned, image-based PDF documents into clean, processable text before the NLP steps begin.
    - **Validation and Heuristic Logic:** Implementing post-processing rules to ensure extracted entities are logically sound (e.g., "The Termination Date cannot precede the Effective Date").

Implementation Details:

- **Core Stack:** TensorFlow/Keras or **Spacy** $\rightarrow$ Tesseract OCR $\rightarrow$ Python.
- **Resources:** Spacy (The industrial-strength NLP library), `pdf2image` (for converting PDF pages to images for OCR).

Week-Wise Implementation Plan:

Week	Focus Area	Tasks/Goals	Review/Validation
Week 1	**OCR Pipeline & Annotation**	Goal: Data preparation for training. Tasks: Build the robust OCR pipeline. Annotate a representative, small subset of contracts manually using a tool like **Doccano** or **Prodigy** (labeling tags like `B-PARTY`, `I-PARTY`, `B-AMOUNT`).	**Text Quality Check:** Evaluate the accuracy of text extraction from noisy or low-quality scanned documents.

Week 2	**NER Modeling**	Goal: Contextual understanding of text. Tasks: Train a custom **Spacy NER** model or a deep learning Bi-LSTM sequence model in TensorFlow using the annotated data.	**Evaluation Metric:** Calculate the **F1-Score** specifically for the most critical entities (Date and Money).
Week 3	**Rule-Based Layer & Precision**	Goal: Maximizing precision and handling edge cases. Tasks: Implement heuristic and constraint-based rules to clean up the model's output (e.g., Standardizing all date formats to ISO 8601).	Comprehensive unit tests for handling **edge cases** and noisy text from imperfect scans.
Week 4	**Containerization & Deliverable**	Goal: Deployable, self-contained service. Tasks: **Dockerize** the entire application (including the model and OCR dependencies). The resulting API must accept a PDF file and return a structured **JSON** object of all extracted entities.	**End-to-End Test:** Run the entire pipeline on a previously "held-out" contract document for final acceptance.

-----4. Project 3: Retail (Computer Vision) - Visual Quality Control System

Project Title: Visual Quality Control System
Product Brand Name: "VisionSpec QC"

Use Case (Production Context): An assembly line for Printed Circuit Boards (PCBs) requires 100% inspection for soldering defects. A high-speed camera captures images. The AI system must classify the image as **"Pass" or "Defect"** and, crucially, visually indicate the defect location—all in **real-time** (low latency).Key Product Features & Deliverables:

- **Basic Requirement:** Build a foundational Convolutional Neural Network (CNN) from

scratch to demonstrate core vision understanding.

- **Deep Production Features:**
  - **Transfer Learning:** To achieve high accuracy with limited industrial defect data, you must fine-tune a powerful, pre-trained network like **MobileNetV2** (optimized for speed) or **ResNet50** (optimized for depth) by replacing and retraining the final layers.
  - **Data Augmentation:** Implement advanced, real-time data augmentation (rotation, zooming, brightness shifts) using **ImageDataGenerator** to dramatically increase the dataset size and make the model robust to variations in lighting and camera angle.
  - **Grad-CAM (Visualization):** Mandatory use of **Gradient-weighted Class Activation Mapping (Grad-CAM)** to generate a **heatmap** overlaying the original image. This proves the model is focusing on the actual defect and not spurious background features.

Implementation Details:

- **Core Stack:** TensorFlow/Keras $\rightarrow$ **OpenCV** (for image manipulation).
- **Resources:** Pre-trained weights for MobileNetV2.

Week-Wise Implementation Plan:

Week	Focus Area	Tasks/Goals	Review/Validation
Week 1	**Image Pipeline & Preprocessing**	Tasks: Load the image dataset. Implement the **ImageDataGenerator** for real-time augmentation. Standardize image resizing and pixel normalization (0-1).	**Visual Review:** Visually confirm that the augmented samples are realistic and cover the necessary variation.
Week 2	**Transfer Learning Model Training**	Tasks: Load a pre-trained base model (e.g., MobileNetV2), freezing its early layers. Add custom Dense layers for the binary classification task. Train the model, preferably utilizing a GPU environment	**Performance Check:** Analyze the Learning Curves (Loss vs. Epochs) to diagnose and mitigate overfitting or underfitting.

		(Google Colab/Kaggle).	
Week 3	**Validation & Explainability (Grad-CAM)**	Tasks: Implement the **Grad-CAM** algorithm. Apply it to misclassified images and correct predictions. Validate that the resulting heatmap correctly highlights the defect area (soldering joint).	**Transparency Check:** Verify that the model is making decisions based on visually logical features, moving beyond the "black box" stage.
Week 4	**Inference Engine & Demo**	Tasks: Optimize the model using techniques like TensorFlow Lite or quantization for fast inference. Write a script to simulate processing a live video stream (frame-by-frame processing).	**Live Demonstration:** Successful execution of the quality control system on new images with low latency.

-----5. Project 4: Telecom (Big Data) - Scalable Customer Churn Prediction

Project Title: Scalable Customer Churn Prediction
Product Brand Name: "SparkScale Churn"

Use Case (Production Context): A major telecommunications provider generates approximately **2 Terabytes (TB)** of log data monthly (call records, data usage, customer complaints). Standard tools like Pandas are incapable of loading or processing this volume, resulting in catastrophic memory errors. The task is to build a distributed Machine Learning pipeline to accurately predict which high-value users are likely to churn (leave) in the next 30 days.Key Product Features & Deliverables:

- **Basic Requirement:** Understand why aggregation in single-machine tools is impossible at this scale.

- **Deep Production Features:**
  - **Distributed Processing Core:** Mandatory use of **PySpark** to ingest, clean, and feature engineer the large dataset by distributing the workload across a simulated compute cluster (using a local setup via Docker).
  - **Spark MLlib Utilization:** Training high-performance models like **Distributed Random Forests** using Spark's native **MLlib** library, which is optimized for parallel computation.
  - **Pipeline Persistence:** Saving the entire data transformation sequence, including the feature engineering steps and the final model, as a single, reusable **Spark ML Pipeline** object for easy deployment in a production environment.

Implementation Details:

- **Core Stack: Apache Spark (PySpark)** $\rightarrow$ Spark SQL $\rightarrow$ Spark MLlib.
- **Resources:** PySpark documentation, Docker for setting up a single-node Spark master/worker environment.

Week-Wise Implementation Plan:

Week	Focus Area	Tasks/Goals	Review/Validation
Week 1	**Spark Setup & Distributed ETL**	Tasks: Set up a local Spark environment (e.g., via Docker Compose). Ingest large CSV/Parquet files into Spark DataFrames. Perform initial cleansing (handling nulls, schema validation).	Validate the ingested data: Confirm accurate row count and schema correctness using PySpark APIs.
Week 2	**Feature Engineering at Scale**	Tasks: Use **Spark SQL** and DataFrame operations to calculate user aggregates (e.g., Average Daily Data Usage, Count of Total Complaints in last 90 days). Use **VectorAssembler** to combine features for	**Optimization Check:** Analyze the Spark Execution Plan (DAG) to ensure efficient, optimized operations and avoid shuffles.

		modeling.	
Week 3	**Distributed Modeling**	Tasks: Train a simple Logistic Regression and a more powerful **Random Forest** using the `pyspark.ml` package. Use a **BinaryClassificationEvaluator** for metric calculation.	**Performance Analysis:** Benchmark and analyze the distributed training time compared to expected sequential processing time.
Week 4	**Production Pipeline Persistence**	Tasks: Build the final **Spark Pipeline** object (e.g., StringIndexer $\rightarrow$ VectorAssembler $\rightarrow$ Model). Save the entire pipeline artifact to disk. Write a batch inference script that loads the saved pipeline.	**Simulated Batch Run:** Test the end-to-end pipeline on a new, unlabelled dataset, simulating a "Daily Batch Churn Scoring" job.

-----Submission Requirements

All deliverables must strictly adhere to the following standards:

1. **Version Control:** All source code **must** be managed and version-controlled on a private/public **GitHub repository**.
2. **Dependencies:** A complete and accurate `requirements.txt` file is mandatory to ensure environment reproducibility.
3. **Documentation:** Clear instructions on how to set up, run, and test the project must be included in a `README.md` file.*From*